Inheritance

Inheritance

Inheritance = mechanism to create a **new class** that **reuses** and **extends** an **existing class.**

- **New class** is also known as **child / derived / subclass**
- **Existing class** is also known as **parent / base / superclass**

- **Child class** inherits **attributes** & **methods** of parent.

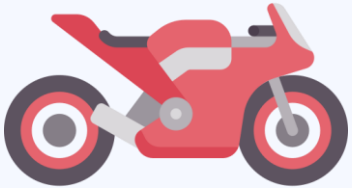
- Use when there is an **"IS A" relationship**:

Benefits:
- Code reuse (no rewriting common logic)
- Consistency (shared behavior in one place)
- Extensibility (override or add new behavior)

Inheritance Example

AUTOMOBILE

Car



Bike

Bus

Types of Inheritance

Inheritance can be divided in **five** different types in python -

1. Single Inheritance
2. Multiple Inheritance
3. Multilevel Inheritance
4. Hierarchical Inheritance
5. Hybrid Inheritance

Single Inheritance

This is the simplest form of inheritance where a **child class** inherits **attributes** and **methods** from **only one parent class**.

Python

```python
class Parent:
    # attributes / methods
    ...

class Child(Parent):
    # extra / overridden behavior
    ...
```

Single Inheritance - Example

Python

```python
class Animal:
    def speak(self):
        print("Some sound")

class Dog(Animal):
    pass
```

Multiple Inheritance

In **Multiple inheritance**, child class is constructed from **more than one parent classes**, and thus inherits the **attributes and method** from all parents.

Python

```python
class Parent1:
    # attributes / methods
    ...

class Parent2:
    # attributes / methods
    ...

class Child(Parent1, Parent2):
    # extra / overridden behavior
    ...
```

Multiple Inheritance - Example

Python	Usage & Output

`class Dad:`
`  def dad_trait(self):`
`  print(“Hardworking Quality")`
 	
`class Mom:`
`  def mom_trait(self):`
`  print(“Very Smart")`
 	
`class Kid(Dad, Mom):`
`  pass`
	`d = Kid()`
	
	`d.dad_trait()`
	`# Hardworking Qualtiy`
	
	`d.mom_trait()`
	`# Very Smart`

Multilevel Inheritance

In **multilevel inheritance**, a class is derived from **another derived class**.

Python

```python
class GrandParent:
    # attributes / methods
    ...

class Parent(GrandParent):
    # extra / overridden behavior
    ...

class Child(Parent):
    # extra / overridden behavior
    ...
```

Multilevel Inheritance - Example

Python	Usage & Output

`class Animal:`
`  def eat(self):`
`  print("Animal is eating")`
 	
`class Dog(Animal):`
`  def bark(self):`
`  print("Dog is barking")`
 	
`class Puppy(Dog):`
`  def play(self):`
`  print("Puppy is playing")`
	`p = Puppy()`
	
	`p.eat()`
	`# Animal is eating`
	
	`p.bark()`
	`# Dog is barking`
	
	`p.play()`
	`# Puppy is playing`

Hierarchical Inheritance

In **hierarchical inheritance**, multiple **child classes** inherit from the **same parent class**.

Python

```python
class Parent:
    # attributes / methods
    ...

class Child1(Parent):
    # extra / overridden behavior
    ...

class Child2(Parent):
    # extra / overridden behavior
    ...
```

Hierarchical Inheritance - Example

Python	Usage & Output

`class Animal:`
`  def eat(self):`
`  print("Animal is eating")`
 	
`class Dog(Animal):`
`  def bark(self):`
`  print("Dog is barking")`
 	
`class Cat(Animal):`
`  def meow(self):`
`  print("Cat is meowing")`
	`d = Dog()`
	`c = Cat()`
	
	`d.eat()`
	`# Animal is eating`
	`d.bark()`
	`# Dog is barking`
	
	`c.eat()`
	`# Animal is eating`
	`c.meow()`
	`# Cat is meowing`

Hybrid Inheritance

Hybrid inheritance is a combination of **more than one type** of inheritance

Python

```python
class Parent:                    # <----base class
    # attributes / methods
    ...

class Child1(Parent):            # <----single / multilevel / hierarchical part
    # extra / overridden behavior
    ...

class Child2(Parent):            # <----another child of Parent (hierarchical)
    # extra / overridden behavior
    ...

class HybridChild(Child1, Child2): # <----hybrid
    # extra / overridden behavior
    ...
```

Hybrid Inheritance - Example

Python	Usage & Output

`class Person:`
`  def show_person(self):`
`  print("I am a person")`
 	
`# Hierarchical inheritance (two children from Person)`
`class Student(Person):`
`  def show_student(self):`
`  print("I am a student")`
 	
`class Teacher(Person):`
`  def show_teacher (self):`
`  print("I am a teacher")`
 	
`class Assistant(Student, Teacher):`
`  def show_assistant(self):`
`  print("I am an assistant")`
	`ta = Assistant()`
	
	`ta.show_person()`
	`# I am a person`
	
	`ta.show_student()`
	`# I am a student`
	
	`ta.show_teacher()`
	`# I am a teacher`
	
	`ta.show_assistant()`
	`# I am an assistant`

super() in Inheritance

super() is used in a **child class** to call **parent methods** (commonly **__init__**):

Python

```python
class Parent:
    def __init__(self, msg):
        self.msg = msg

class Child(Parent):
    def __init__(self, msg):
        super().__init__(msg)      # calls Parent.__init__

c = Child("Hello")
print(c.msg)    # Hello
```

WATCH

Level up your coding with each episode in this focused Python series.

Next Video!

Abstraction